frameworks such as Angular and React. Your .NET code is compiled to WebAssembly (https://webassembly.org) or executes on a .NET runtime compiled for WASM, and the browser downloads and runs it as it would a JavaScript app. This way you can build highly interactive client-side applications while using C# and all the .NET APIs and libraries you already know.

By contrast, Blazor Server applications run on the server. Each mouse click or keyboard event is sent to the server via WebSockets. Then the server calculates the changes that should be made to the UI and sends the required changes back to the client, which updates the page in the browser. The result is a "stateful" application that runs server-side but can be used to build highly interactive SPAs. The main downside of Blazor Server is that it requires a constant internet connection.

NOTE In this book I focus on building traditional page-based, server-side-rendered web applications and RESTful web APIs. I also show how to create background worker services in chapter 34. For more information on Blazor, I recommend *Blazor in Action*, by Chris Sainty (Manning, 2022).

With the ability to call on all these paradigms, you can use ASP.NET Core to build a wide variety of applications, but it's still worth considering whether ASP.NET Core is right for your specific application. That decision will likely be affected by both your experience with .NET and the application you want to build.

2.4 When to choose ASP.NET Core

In this section I'll describe some of the points to consider when deciding whether to use ASP.NET Core and .NET 7 instead of legacy .NET